

Refutation-gated admission

Integrity for agents that do not repeat.

Roque Briceño

Version 0.8.0. 30 August 2026. Licensed under CC BY 4.0.

Abstract

Fail-closed class dispatch proves that a work item's stage ran on the specialist and model it was bound to. With a deterministic function, knowing which function ran says what came out. With a language model it says almost nothing: the same specialist, prompt and context produce different artifacts, and the dispatch passes all of them identically. FCD verifies the die, not the roll. Its own proofs list the quality and provenance of a passed body as unproved. What this paper proves is integrity of the refutation record, not truth of any claim.

Refutation-gated admission moves the gate from the binding to the claim. A class fixes, before any artifact exists, the formal claims an artifact must satisfy and the deterministic refuters that will attack them. A refuter is versioned, replayable, declared as authored by neither the generator nor the defect model's author (author identities are strings the kernel compares, B11), and carries a power as a labelled record, never a bare figure: a kill-rate the kernel counts from a ledger of seeded defects against a named defect model, or $1-(1-\epsilon)^N$ from a declared (ϵ, N) the seal carries. The same bind is run k times; a sample is an FCD-passed stage. Every refuter is tried on every sample. A trial is refuted, survived, or inconclusive; only survived counts. Admission requires every cell survived, every refuter replayed once with the same outcome, concordance with the designated sample at or above a declared threshold, and power at or above a declared floor. The seal carries the artifact hash, the sampling configuration, the power per claim with its defect model or its (ϵ, N) , the concordance as (agreeing, k), and the class's declared list of what was not attacked. Nothing raises a sealed power afterwards.

This is mutation analysis, hermetic replay and semantic entropy applied as guards on an FCD line, not a new integrity model. Clark and Wilson already have the integrity verification procedure and the certification rules; FCD modelled the enforcement half and left the IVP out. RGA models the IVP and its certification record, and says which half is enforced and which is certified out of band. The theorems are about the refutation record — that this refuter version, at this measured power, ran on these bytes under this seed and fuel, and a third party can re-derive the verdict. They are not about the truth of the claim. RGA extends FCD by composition: it reads FCD state, writes none of it, and inherits I1, I2, I3, I5 and I8 by citation; the check stage's dual control (I6) is inherited unchanged and no R-proof cites it.

1. Problem

An agent that does not repeat cannot be certified by naming it. Two runs of one bind differ; a gate that passes on identity passes both the right artifact and the wrong one, and publishes the same green. Test suites distinguish them, but a green from an empty suite and a green from a suite that kills ninety-five per cent of seeded defects look identical at the merge. Reviewers distinguish them, but a reviewer is another agent that does not repeat. Self-consistency votes among samples, and a vote is a tiebreak that picks a winner rather than a precondition that refuses.

The required property: nothing is admitted except through a declared refutation that was given a fair chance to kill the claim and failed; the seal records how strong that chance was; and the generator's own instability on the claim is a fault, not a nuisance.

Three inversions, each with prior art named in §10:

1. The generator is untrusted and stochastic; the refuter is deterministic and replayable. Integrity attaches to the refutation record, the only part that can be re-executed to the same answer.
2. Acceptance carries measured power against a declared alternative. A survival at 0.2 and a survival at 0.98 cannot share a seal; neither can a survival against a small adversarial defect model and one against a large redundant one, because the seal names the model.
3. Variance is a fault. The same bind is sampled k times; disagreement on the claim — not the prose — closes the gate.

PREMISE.md records the attack on those inversions before anything was built. The second survives with *measured* relativised; the fifth proposed theorem, that determinism is checkable, was restated as an assumption plus a falsifier.

2. Objects

A **line** is one FCD work item whose first k stages are write stages. Its **bind key** is $(c, \text{body}, \text{FCD policy version}, a, \text{norm}(\phi(a)), \sigma)$ with σ the sampling configuration. A **sample** is stage $i < k$ after FCD Pass with the line's bind key; its artifact is hashed by the kernel over the bytes it is handed, and a nonce is drawn after the hash. Sample 0 is **designated**: it is stage 0, fixed before any sample exists.

A **claim** is a formal statement over a fixed artifact format, named by a spec hash. A class pins its claims and, per claim, the refuters that attack it and the defect model they are measured against. The generator does not author the claim it must meet.

A **refuter** $r = (\text{id}, \text{version}, \text{author}, \text{mode})$ is deterministic under B2 and replayable. Mode *ledger*: measured against a defect model. Mode *bounded*: a seeded sampler with a declared (ϵ, N) .

A **defect model** $D = (\text{hash}, \text{author})$ names a finite set of seeded defects, each with a killing witness so that none is equivalent to the original (B5).

A **power record** $P[(r, D)]$ is write-once. Ledger: the kernel counts kills over a ledger of per-defect verdicts; inconclusive entries count in the denominator and not the numerator; power is $\text{kills}/|D|$, exact on D . Bounded: power is $1 - (1-\epsilon)^N$, the standard bound for any defect of failure mass at least ϵ under the declared distribution. No interval is carried: a binomial interval assumes i.i.d. draws from the population the seal speaks to, and a mutant set is not that.

A **trial** is one refuter against one claim on one sample, with a seed the kernel derives from the sample's nonce and artifact hash, and a verdict in $\{\text{refuted}, \text{survived}, \text{inconclusive}\}$ plus a witness hash — the canonical form of the claim-relevant observable the refuter computed.

Concordance for a claim is $\text{agreeing}/k$ where agreeing counts samples whose witness vector on that claim equals sample 0's. Compared against the designated sample, never a plurality.

A **seal** is the record written to S_R (§5).

3. Process

1. Declare refuters. Measure each against the class's defect model from a ledger, or bound it from (ϵ, N) . Records are write-once.

2. Open an FCD work item; open the line on it before any sample stage is attempted. The class's claims and refuters are pinned. A refuter declared after this point, authored by the generator, or refused, cannot be pinned.
3. For $i < k$: run the FCD write stage (Admit, Bind, Observe, Pass — I1 holds of it); hand the kernel the artifact bytes and the package categories; the kernel hashes, draws the nonce, and registers the sample. A package that carried refuter material is refused.
4. For every (claim, refuter, sample): obtain the seed from the kernel; run the refuter under B2; report the verdict and witness. Refuted or inconclusive closes the line, published.
5. Replay at least one trial per refuter with identical outcome. A divergence refuses the refuter for every line.
6. Run the FCD check stage, if the class has one (I6 excludes the generator). FCD Accept writes S.
7. Seal. Requires every cell survived, every refuter replayed and measured, the defect model independent of generator and refuter, $id \in S$, concordance $\geq \theta$ on every claim, and $\min_j \text{composite}_j \geq p_{\min}$. Below θ or $p_{\min}$ the line closes with V2 or V5; it does not choose a better sample or a better number.

4. What holds

Machine: rga/core.py. Proofs: PROOFS.md. Checks: tests/test_rga_invariants.py, tests/test_rga_mutation.py, tests/test_rga_citations.py.

Under B0–B11, R1–R13 are inductive on the combined machine. An independent five-lens review (round 1) broke three of the first proofs — a defect-model author free per record, a caller-supplied journal position, an unreplayable discord close — and the repairs are cited in PROOFS.md. R1: no admission without survival in every cell. R2: power is carried from a write-once record the kernel computed, never inferred. R3: the refuter was fixed before generation, not authored by the generator, and measured against a defect model authored by neither. R4: concordance with the designated sample is a precondition. R5: a replay divergence refuses the refuter monotonically. R6: every refuter was replayed at least once. R7: every sample satisfies I1, I2, I3 — by citation. R8: $S_R \subseteq S$, hence I5, I8. R9, R10: trials are bound to kernel-hashed bytes through a seed that cannot exist before the artifact. R11: RGA writes no FCD field; the FCD proofs hold on the combined machine because its FCD projection is the FCD machine. R12, R13: frozen line, bounded samples and trials, write-once power.

Every guard the fault table names is a named method. tests/test_rga_mutation.py replaces each with a no-op — one at a time — and shows the forbidden state becomes reachable; a guard method no deletion turns red fails the suite as dead code. Transition-shape preconditions (wrong pc, unknown ids, malformed verdicts, an index outside the sample range) are inline raises witnessed by the invariant tests. tests/test_rga_citations.py opens every file:line:symbol in the proofs and fails if the line no longer contains the symbol — a move-detector; the proofs' sentences, not that test, claim enforcement.

5. The seal

$S_R[id]$ carries: the designated sample's artifact hash; k , θ , $p_{\min}$, the sampling configuration, the RGA policy version; the FCD identity (class, body, specialist, executed model, FCD policy version); per claim, each refuter as (id, version, mode, power, D, kills, |D|, ϵ , N), the composite with its label — *single*, *union* over a shared D, or *max* — and (agreeing, k); $\min_j \text{composite}_j$; the residual — the

class's declared not-attacked intents, whose *check_stage* disposition is refused unless an FCD check stage Passed; and the journal position.

A seal is a record. A later refusal of a refuter does not rewrite it; tainted is a pure query. A deployment's DAG edge and promotion predicate must read S_R with a floor; one that leaves them reading S has FCD's guarantee at the edge, not this one.

6. Faults

Each is a forbidden transition, in its own namespace (V) so fault codes never collide with theorem numbers (R). V1–V5 are observed and publish a close; V6–V15 are guard refusals that write nothing.

V1 — Seal after a trial on the line returned refuted

V2 — Seal with agreeing $j/k < \theta$ on any claim

V3 — Seal counting an inconclusive trial as survival

V4 — Agreement recorded for a replay that diverged; Measure, Bound or Replay on a refused refuter

V5 — Seal with $\min_j \text{composite}_j < p_{\min}$

V6 — Sample from a stage not Passed or bound to a different specialist than the line

V7 — Sample whose generator package contained a refuter category

V8 — Open after a sample stage was attempted; a sample registered after a later sample stage was attempted; Trial against an unpinned claim or refuter

V9 — Trial whose seed is not the kernel-derived seed

V10 — Seal without k samples or with a cell lacking a surviving trial

V11 — Seal using a refuter with no identical replay on this line

V12 — Seal using a refuter with no power record

V13 — Measure or Bound on an existing key; a sample beyond k; a second trial in one cell; a ledger whose id-set, or a record whose author, differs from the first record

V14 — A pinned refuter authored by the generator; a defect model authored by its recording refuter, or whose fixed author is the generator

V15 — Seal of a line not in S; a residual claiming a review that did not run; any write to S_R but Seal

7. The boundary

A claim is RGA-attackable iff its falsity has a finite witness that a total, fuel-bounded, hermetic, replay-stable checker accepts. That line is intrinsic to a formal claim. What is relative is the gap between an intent and the formal claims that stand in for it: "this is the correct fix" has the attackable shadow "the issue no longer reproduces, the held-out tests pass, the diff touches only the named files", and the residual "that the held-out tests test the issue". Every interesting intent has only a shadow. A

seal that attacked the shadow at power 1 must not be read as a verdict on the intent, so the seal lists what it did not attack and who, if anyone, looked at it.

A language model can propose counterexamples. A checker-verified witness is a sound, replayable kill whoever proposed it. A survival under proposed attack carries power only if the proposer is pinned and its catch rate measured on a held-out seeded set, the rule for every stochastic witness source. The model is never the refuter.

Prose is not excluded by medium. Perturbation families for faithfulness exist, and fixed-weight consistency checkers that are bitwise replayable when pinned report agreement with human-labelled errors in the 0.7–0.85 range (balanced accuracy on the published benchmarks — not a kill-rate, which a deployment would measure against its own D); omission and adequacy defects have near-zero measured power today, and a seal over them says so.

Determinism is not checkable; it is assumed by construction (B2: pinned toolchain, no clock or network, runtime-counted fuel — refusing a refuter written outside that subset is the harness's obligation, not a kernel check) and falsified by replay. Exhaustion is a third outcome that never counts as survival. The artifact runs inside the refuter's sandbox; the refuter's parser is a trusted computing base against hostile input.

8. Measurement

Rates on a named cut, as in FCD §7, never theorems: refutation rate per class and refuter version; discord rate at the declared (k, σ) ; *miss observed* per refuter — where the witness is a function of the claim value, a discordant pair of survivals is a detected miss by that refuter, and a V2 close journals exactly the refuters whose witnesses differed; below $\theta = 1$ a discordant survival can seal, visibly, through (agreeing, k); replay divergence per refuter version, the refuter analogue of misbind; escape replay for a successor refuter version against defects later found under its predecessor, informative only across versions and doubly selected. Schema: `../metrics/SCHEMA.md`. No numbers here.

9. Calibration — the escape ledger

B6 leaves the defect model's resemblance to the generator's real defects an assumption, and nothing above forces a found miss to have any consequence. The calibration layer (`rga/calibration.py`, theorems C1–C7, faults E1–E9, INVARIANTS.md §8) closes the loop that can be closed and says plainly which loop cannot. "Calibration" is this repository's metrological sense: the reference corpus is ratcheted against verified real defects. No statistical sense is claimed.

An **escape** is a counterfactual trial: the sealed claim's *pinned* refuter, run at a finder-chosen nonce against the sealed bytes — the finder authors nothing but the nonce, the kernel hashes the bytes and derives the seed itself, and the verdict refuted plus one identical replay establishes it. That channel adds no assumption the seal did not already carry. Any other checker is tier B and has effect only through a journaled, attributed adjudication — the per-escape claim-fidelity judgment made visible instead of pretended away. A **charge** is the wrong-verdict cell (line, claim, refuter version), one per cell however many witnesses prove the miss. **Impeachment** is a pure query entailed by a valid escape — the seal is immutable, revocation is consulted at use, and it cannot be declined: unlike a certificate authority's revocation list, the entry is entailed by a replayable proof, and the miss propagates mechanically to every checker that vouched. The **ratchet** refuses any successor policy whose defect models forget a valid escape without a named, journaled exclusion, and the install event — which now exists — carries the coverage primaries and the dropped-id diff. **Demotion** is a query, never a cascade: charges past a declared budget block new pins, and gate Seal exactly where the class declared it, closing with a published fault that carries the instrument's primaries. Every seal issued through the

authority is stamped, in the same step, with each pinned refuter's track record — primaries, never a rate; a zero is absence of filed evidence and the record says so.

What this proves is one sentence shorter than it sounds: **forgetting is loud**. Every found miss *that stays in the record* has a mandatory, attributed, replayable consequence — and the qualification is load-bearing: replay re-derives every transition and compares it to the journal field by field, so alteration, forgery and duplication are refused, but a deletion that leaves a coherent shorter history is not detectable from the journal alone, and that is the direction which raises standing (§11). What was never found, the ledger cannot see — the corpus is a selected, steerable subsample of real defects, silence is unreadable except through journaled audits whose reference is the corpus itself, and every aggregate an adversary can steer by filing only true things. The C-theorems are about the ledger of found misses, never the distribution of real ones, and reading seven theorems under a calibration heading as "the coupling gap is closed" is exactly the laundering the seal's power field was built to prevent. Premise round and its one authored adjudication:
`eval/reviews/rga-calibration-premise-SYNTHESIS.md`.

10. Related work

Clark and Wilson (1987): certification rules and the integrity verification procedure. FCD's specialist is a TP; RGA's refuter is an IVP; its power is a certification made by a party with I/O, and the kernel enforces that the certified IVP of the pinned version ran and that its record is on the seal.

Mutation analysis (DeMillo, Lipton & Sayward 1978; Jia & Harman 2011) owns the ledger number. Just et al. (2014) measured its coupling to real faults; Papadakis et al. (2016) its threats to validity; Kurtz et al. (2014) dominator mutants. Property testing (Goldreich, Goldwasser & Ron 1998), Freivalds (1977) and Miller–Rabin own the bounded number. Proof-carrying code (Necula & Lee 1996) is the power-1 corner and the seal still carries the checker's identity there.

Semantic entropy (Kuhn, Gal & Farquhar 2023; Farquhar et al. 2024) is the graded statistic of which concordance is the zero-entropy special case — exact agreement with a designated sample, as a gate rather than a signal; self-consistency (Wang et al. 2022) is the vote RGA refuses to take; AlphaCode's output clustering, CodeT and universal self-consistency are agreement on claim consequences without an oracle. N-version programming (Avizienis 1985) and Knight & Leveson (1986) are why power does not compose by independence. Flaky-test quarantine (Luo et al. 2014) and reproducible builds are the practice behind R5. In-toto (Torres-Arias et al. 2019) is the provenance the trial record resembles; what it lacks is concordance under a claim normaliser and a typed store a DAG edge reads. Statistical model checking (Younes & Simmons 2002) is the formal home for a stamped survival rate with error bounds. Conformal factuality (Mohri & Hashimoto 2024) is concordance plus calibration with no refuter. Metamorphic testing (Chen, Cheung & Yiu 1998) and FactCC (Kryscinski et al. 2020) supply claims and defect models where no oracle exists. Automated-repair overfitting (Smith et al. 2015) is why the generator's package excludes the refuter and why the seed is drawn after the artifact.

The calibration layer's loop has its own ancestry: eliminative argumentation and Assurance 2.0 defeater discipline (Goodenough, Weinstock & Klein 2015; Bloomfield & Rushby 2024) — confirmed, refuted, or accepted with recorded rationale, never silently dropped — mechanized, with the non-covering successor refused by a kernel rather than flagged for an assessor. The corpus is escape-to-test regression practice and mined-from-real-faults mutation (Tufano et al. 2018; Beller et al. 2021; Defects4J) applied as a journal-derived ratchet. Demotion is flaky-test quarantine and the SRE error budget with the sign flipped — those practices lack a verified false-negative channel, which is what the escape object creates — and browser root-program distrust of certificate authorities is the institutional precedent. Impeachment is the CRL/OCSP/Sigstore shape minus discretion: the two properties for which no prior formalization was found are non-discretion (revocation entailed by a replayable proof)

and charge totality (mechanical consequence for every vouching checker). Adaptive conformal inference (Gibbs & Candès 2021) is the shape a statistical calibration theorem has, and why none is claimed here.

The 2026 neighbours circle the same gap without the enforcement object. Refute-or-Promote (arXiv:2604.19049) puts adversarial kill mandates at promotion gates, but its refuters are LLM agents — stochastic, unreplayable, with no carried power. ConsistencyGate (arXiv:2607.22962) is admission control by self-consistency, but as a thresholded soft score — a vote, where concordance here is a fault against a designated sample. Attestation-aware promotion gates (FSE 2026, arXiv:2603.28988) bind provenance of training and release claims — the B1/B2 boundary, complementary, carrying no power. And a contract-grade verifier for generated GPU kernels (arXiv:2608.12700) is the premise measured in one domain: the field's loose test accepted 1,487 kernels the rigorous verifier rejected, against 14 the other way — two greens that today print identically, which is what the seal's carried power exists to separate. Mutation-kill certification is reported absent from LLM generation benchmarks (arXiv:2608.12635). What none of these carry is the seal: kernel-counted power against a named defect model, concordance as a precondition, refuter refusal on replay divergence, and a proved fail-closed table underneath.

11. Limits

Record completeness. Replay proves the events present form a history the live machine would have accepted, not that it is the history that actually ran. A coherent rewrite of root transition inputs is another valid history; deletion — the journal's tail, or any coherent group no surviving event recomputes against — is likewise undetectable from replay alone and can raise standing. v0.5 adds an authenticated publication path: each authority owns an immutable tuple journal, each stack has a stable registry namespace, successor heads prove prefix extension through a cumulative event chain, three heads advance atomically, and a signed composed receipt derives its predicates from the exact current I/R/C state. After a trusted first anchor or continuous publication, verification rejects truncation, coherent deletion, longer forks, state-state receipts, namespace collisions, and cross-wired stacks. *Stale* here means not registry-current for the current journal state; *issued_at* is signed metadata, not expiry. The first receipt cannot distinguish coherent histories that predate its trusted bootstrap. The closure is conditional on using that path, HMAC-SHA256 unforgeability, SHA-256 collision and second-preimage resistance, honest key custody, and durable registry state; registry plus signer/key compromise or rollback and mutation outside the atomic single-writer authority model remain assumptions. **Mediation.** A seal produced by calling `Admission.seal` directly is layer R only: `mediated(id)` — one stamp bound to that seal — is a conjunct of admissible, so such a line reads IR, not IRC. **Budgets.** `e_max` and `demotion_gate` are explicit per class (E9); an unconfigured class is refused, never treated as unlimited, and the budgets are journaled so a rebuild runs under the ones the record names. No dataset. No quality theorem. No liveness. R1–R13 hold on the combined Python machine under B0–B11; B1 (harness honesty) and B2 (hermetic runtime) are physical-boundary assumptions of A10's standing, and author identities are declared strings (B11), not a closed namespace as FCD's specialist ids are. Power is relative to a declared alternative the seal names; its relevance to the generator's actual defects is the coupling-effect hypothesis, stated, not proved. Claim fidelity to intent is not proved. Body provenance is not closed; it is moved to the seal's hash and still rests on I1 holding of the report. A refuter the generator already knows can be special-cased; the seed and the package exclusion narrow that and do not close it. The FCD check stage is inherited as a dispatch-integrity gate and carries no content authority.

"Subsumes" is by composition. RGA extends FCD, writes no FCD field, and needs FCD underneath as a separate, cited layer.

Site names do not belong in this voice.

12. References

- Avizienis, A. The N-version approach to fault-tolerant software. IEEE TSE, 1985.
- Chen, T. Y., Cheung, S. C., and Yiu, S. M. Metamorphic testing. HKUST-CS98-01, 1998.
- Chen, B. et al. CodeT: Code generation with generated tests. 2022.
- Chen, X. et al. Universal self-consistency for large language model generation. arXiv:2311.17311, 2023.
- Clark, D. D., and Wilson, D. R. A comparison of commercial and military computer security policies. IEEE S&P, 1987.
- DeMillo, R. A., Lipton, R. J., and Sayward, F. G. Hints on test data selection. IEEE Computer, 1978.
- Farquhar, S. et al. Detecting hallucinations in large language models using semantic entropy. Nature, 2024.
- Freivalds, R. Probabilistic machines can use less running time. IFIP, 1977.
- Bloomfield, R., and Rushby, J. Defeaters and eliminative argumentation in Assurance 2.0. arXiv:2405.15800, 2024.
- Goodenough, J., Weinstock, C., and Klein, A. Eliminative argumentation: a basis for arguing confidence in system properties. SEI, 2015.
- Gibbs, I., and Candès, E. Adaptive conformal inference under distribution shift. NeurIPS, 2021.
- Tufano, M. et al. Learning how to mutate source code from bug-fixes. arXiv:1812.10772, 2018.
- Beller, M. et al. What it would take to use mutation testing in industry — a study at Facebook. ICSE-SEIP, 2021.
- Goldreich, O., Goldwasser, S., and Ron, D. Property testing and its connection to learning and approximation. JACM, 1998.
- Jia, Y., and Harman, M. An analysis and survey of the development of mutation testing. IEEE TSE, 2011.
- Just, R. et al. Are mutants a valid substitute for real faults in software testing? FSE, 2014.
- Knight, J. C., and Leveson, N. G. An experimental evaluation of the assumption of independence in multiversion programming. IEEE TSE, 1986.
- Kryscinski, W. et al. Evaluating the factual consistency of abstractive text summarization. EMNLP, 2020.
- Kuhn, L., Gal, Y., and Farquhar, S. Semantic uncertainty. ICLR, 2023.
- Kurtz, B. et al. Mutant subsumption graphs. ICSTW, 2014.
- Li, Y. et al. Competition-level code generation with AlphaCode. Science, 2022.

Luo, Q. et al. An empirical analysis of flaky tests. FSE, 2014.

Mohri, C., and Hashimoto, T. Language models with conformal factuality guarantees. ICML, 2024.

Necula, G. C., and Lee, P. Safe kernel extensions without run-time checking. OSDI, 1996.

Papadakis, M. et al. Threats to the validity of mutation-based test assessment. ISSTA, 2016.

Smith, E. K. et al. Is the cure worse than the disease? Overfitting in automated program repair. FSE, 2015.

Torres-Arias, S. et al. in-toto: Providing farm-to-table guarantees for bits and bytes. USENIX Security, 2019.

Wang, X. et al. Self-consistency improves chain of thought reasoning in language models. 2022.

Younes, H. L. S., and Simmons, R. G. Probabilistic verification of discrete event systems using acceptance sampling. CAV, 2002.

Refute-or-Promote: an adversarial stage-gated multi-agent review methodology for high-precision LLM-assisted defect discovery. arXiv:2604.19049, 2026.

ConsistencyGate: preventing memory contamination in LLM agents via self-consistency admission control. arXiv:2607.22962, 2026.

Attesting LLM pipelines: enforcing verifiable training and release claims. FSE, arXiv:2603.28988, 2026.

A contract-grade verifier for LLM-generated GPU kernels. arXiv:2608.12700, 2026.

GateTruth: auditing the rigor of RTL design benchmarks via mutation testing. arXiv:2608.12635, 2026.